

```

1  #!/usr/bin/env python
2  # coding: utf-8
3
4  import os
5  import json
6  import glob
7  import copy
8  from datetime import datetime, timedelta
9  from dateutil.relativedelta import relativedelta
10 import pickle
11
12 import pandas as pd
13 import numpy as np
14 pd.set_option(
15     'display.max_rows', None,
16     'display.max_columns', None,
17     'display.max_colwidth', 800
18 )
19
20 from pyhive import hive
21 from impala.dbapi import connect
22
23 import psycopg2
24 from sqlalchemy import create_engine
25
26 from f_utils import *
27 import argparse
28
29
30 import trino
31 from trino.auth import BasicAuthentication
32 import warnings
33 from urllib3.exceptions import InsecureRequestWarning
34 warnings.simplefilter('ignore', InsecureRequestWarning)
35
36
37
38 def holo_prod_conn(xdb="features_engineering_ndf2w"):
39     alihost = 'hgpost-sg-4hr2rtzjh002-ap-southeast-5-vpc-st.hologres.aliyuncs.com'
40     aliuser = 'LTAI5tEKBAQHBhm3Z2kaGm7q'
41     alipass = 'SkjYLF98XzjhizVzDwp7QQyWTC9Bry'
42     conn = psycopg2.connect(
43         database = xdb,
44         user      = "{usr}".format(aliuser),
45         password  = "{pss}".format(alipass),
46         host      = "{hst}".format(alihost),
47         port      = 80
48     )
49     return conn
50
51
52 def holo_prod_los():
53     alihost = 'hgpost-sg-4hr2rtzjh002-ap-southeast-5-vpc-st.hologres.aliyuncs.com'
54     aliuser = 'LTAI5tEKBAQHBhm3Z2kaGm7q'
55     alipass = 'SkjYLF98XzjhizVzDwp7QQyWTC9Bry'
56     db_     = 'los'
57     conn_string = 'postgresql+psycopg2://{usr}:{pss}@{hst}:80/{db}'
58     conn_string = conn_string.format(
59         usr = aliuser,
60         pss = alipass,
61         hst = alihost,
62         db  = db_
63     )
64     db = create_engine(conn_string)
65     conn = db.connect()
66     return conn
67

```

```

68
69 def holo_prod_feats():
70     alihost = 'hgpost-sg-4hr2rtzjh002-ap-southeast-5-vpc-st.hologres.aliyuncs.com'
71     aliuuser = 'LTAI5tEKBAQHbhm3Z2kaGm7q'
72     alipass = 'SkjYLF98XzjhizVzDwp7QQyWTC9Bry'
73     db_ = 'features_engineering_ndf2w'
74     conn_string = 'postgresql+psycopg2://{usr}:{pss}@{hst}:80/{db}'
75     conn_string = conn_string.format(
76         usr = aliuuser,
77         pss = alipass,
78         hst = alihost,
79         db = db_
80     )
81     db = create_engine(conn_string)
82     conn = db.connect()
83     return conn
84
85
86 def holo_nonprod_loos():
87     conn = psycopg2.connect(
88         database="loos",
89         user = "LTAI5tEKBAQHbhm3Z2kaGm7q",
90         password = "SkjYLF98XzjhizVzDwp7QQyWTC9Bry",
91         host = "hgpost-sg-4hr2x8isw001-ap-southeast-5.hologres.aliyuncs.com",
92         port = 80
93     )
94     return conn
95
96 # Columns Needed from Raw Contracts
97 contract_cols = [
98     'application_id'
99     , 'ContractList'
100     , 'Creditor'
101     , 'PhaseOfContract'
102     , 'ContractStatus'
103     , 'PastDueDays'
104     , 'OutstandingAmountLocalValue'
105     , 'TotalFacilityAmountValue'
106     , 'WorstPastDueDays'
107     , 'ConditionDate'
108     , 'StartDate'
109     , 'MaturityDate'
110     , 'RealEndDate'
111     , 'DefaultDate'
112     , 'LastUpdate'
113     , 'Pefindo_HistoryID'
114 ]
115
116 # Columns Needed from Raw Payments
117 payment_cols = [
118     'application_id'
119     , 'ContractList'
120     , 'Date'
121     , 'PastDueDays'
122     , 'PastDueAmountValue'
123     , 'PaymentCalendarList'
124     , 'Pefindo_HistoryID'
125 ]
126
127 # Function to explode contractlist and paymentlist raw data
128 def explode_col(df,explode_col : str):
129     list_cols = ['application_id',explode_col]
130     df_exploded = df[list_cols].explode(explode_col,ignore_index=True)
131     json_df = pd.concat(
132     [df_exploded.drop(columns=explode_col), df_exploded[explode_col].apply(pd.Series)],
133     axis=1)
134     return json_df

```

```
135
136
137 user      = 'ram_094151'
138 password='&9OuAc4LCD$IDK%x'
139 database  = 'raw_data_prod'
140
141 trino_conn = trino.dbapi.connect(
142     host='10.121.1.62',
143     port=7778,
144     catalog='hive',
145     schema=database,
146     auth=BasicAuthentication(user, password),
147     http_scheme='https',
148     verify=False
149 )
150
151
152 trino_cursor = trino_conn.cursor()
153
154
155 hive_host_name = "10.121.1.62"
156 hive_port = 10000
157 hive_conn = hive.Connection(
158     host=hive_host_name,
159     port=hive_port,
160     username='ram_094151',
161     password='&9OuAc4LCD$IDK%x',
162     database='raw_data_prod',
163     auth='LDAP')
164
165
166 hive_cursor = hive_conn.cursor()
167
168
169
170 vSQL = """
171 select  count(1) as cnt
172 from    bravo_prod.bpm_s1_outbound_result
173 where  outbound_type in ('PD_1P5C_OVERLAY_NTB','PD_1P5C_OVERLAY_NTC')
174 """
175 features_1p5c = pd.read_sql(vSQL, holo_prod_los())
176 print(features_1p5c)
177
178
179 vSQL = """
180 select  *
181 from    bravo_prod.bpm_s1_outbound_result
182 where  outbound_type in ('PD_1P5C_OVERLAY_NTB','PD_1P5C_OVERLAY_NTC')
183 limit 3
184 """
185 features_1p5c = pd.read_sql(vSQL, holo_prod_los())
186 print(features_1p5c)
187
188
189 vSQL = """
190 select  min(created_at) as min_created_at
191         , max(created_at) as max_created_at
192 from    bravo_prod.bpm_s1_outbound_result
193 where  outbound_type in ('PD_1P5C_OVERLAY_NTB','PD_1P5C_OVERLAY_NTC')
194 """
195 features_1p5c = pd.read_sql(vSQL, holo_prod_los())
196 print(features_1p5c)
197
198
199
200 vSQL = """
201 select  to_char(created_at, 'YYYYMMDD') as created_dtid
```

```

202         , count(1) as cnt
203 from     bravo_prod.bpm_sl_outbound_result
204 where outbound_type in ('PD_1P5C_OVERLAY_NTB','PD_1P5C_OVERLAY_NTC')
205 group by to_char(created_at, 'YYYYMMDD')
206 order by to_char(created_at, 'YYYYMMDD')
207 """
208 features_1p5c = pd.read_sql(vSQL, holo_prod_los())
209 print(features_1p5c)
210
211
212 appsSQL = """
213 with appids
214 as (
215     select  distinct
216             application_id
217     from     bravo_prod.bpm_sl_outbound_result
218     where    ( created_at >= '{lobound}' )
219             and ( created_at <  '{upbound}' )
220             and outbound_type in ('PD_1P5C_OVERLAY_NTB','PD_1P5C_OVERLAY_NTC')
221 )
222 select * from appids
223 """
224
225 vSQL = """
226 with contractlists
227 as (
228     select  id
229             , application_id
230             , created_date
231             , details::jsonb as details
232     from     bravo_prod.bpm_pefindo_detail
233     where    ( details is not null )
234             and ( details like '%%Pefindo_TblContractList%%' )
235             and application_id in ({appids})
236 ),
237 contractlists_
238 as (
239     select  id
240             , application_id
241             , details
242             , created_date
243             , contractlists::json as contractlists
244     from (
245         select  id
246                 , application_id
247                 , details
248                 , created_date
249                 , (details->>'Pefindo_TblContractList'::text) AS contractlists
250         from    contractlists t1
251     ) t2
252     where     contractlists is not null
253 )
254 select * from contractlists_
255 """
256
257 pymtSQL = """
258 with contractpaymentlists
259 as (
260     select  id
261             , application_id
262             , created_date
263             , details::jsonb as details
264     from     bravo_prod.bpm_pefindo_detail
265     where    ( details is not null )
266             and ( details like '%%Pefindo_TblContractPaymentCalendarList%%' )
267             and application_id in ({appids})
268 ),

```

```

269 contractpaymentlists_
270 as (
271     select  id
272            , application_id
273            , details
274            , created_date
275            , contractpaymentlists::json as contractpaymentlists
276     from (
277         select  id
278                , application_id
279                , details
280                , created_date
281                , (details->>'Pefindo_TblContractPaymentCalendarList'::text) AS
282                    contractpaymentlists
283         from    contractpaymentlists t1
284                ) t2
285     where      contractpaymentlists is not null
286 )
287 select * from contractpaymentlists_
288 """
289 dates = [datetime.strptime(datetime(2024,10,1) + relativedelta(months=item*1), '%Y-%m-%d')
290           for item in range(10) ]
291 dates = [(item, datetime.strptime(datetime.strptime(item, '%Y-%m-%d') + relativedelta(
292 months=1), '%Y-%m-%d')) for item in dates]
293
294 container_contract = []
295 container_payment = []
296 for date_ in dates :
297     lobound = date_[0]
298     upbound = date_[1]
299     print(lobound, upbound)
300     appsSQL_ = appsSQL.format(
301         lobound = lobound,
302         upbound = upbound,
303     )
304     appids = pd.read_sql(appsSQL_, holo_prod_los() )
305     print('chunksize : ',appids.shape)
306     if appids.shape[0] > 0 :
307         appids = ','.join(["'{}'".format(item) for item in appids['application_id'].
308 unique()])
309         print(appids[:60])
310         vSQL_ = vSQL.format(
311             appids = appids,
312         )
313         contractlists = pd.read_sql(vSQL_, holo_prod_los() )
314         print('contractlist chunksize : ',contractlists.shape)
315         container_contract.append(contractlists)
316         pymtSQL_ = pymtSQL.format(
317             appids = appids,
318         )
319         paymentlists = pd.read_sql(pymtSQL_, holo_prod_los() )
320         print('paymentlist chunksize : ',paymentlists.shape)
321         container_payment.append(paymentlists)
322
323 contractlists = pd.concat(container_contract)
324 contractlists = contractlists.sort_values(
325     by=['application_id'],ascending=True
326 )
327 contractlists = contractlists.reset_index(drop=True)
328 print('Contractlists Shape : ', contractlists.shape)
329 # contractlists.to_pickle('store/pickle/all_contractlists_adri_v20250730.pkl')
330
331 exploded_contract = explode_col(contractlists,'contractlists')
332 print('Exploded Contract Shape : ',exploded_contract.shape)
333 contract_clean = exploded_contract.drop_duplicates()

```

```
332 contract_clean = contract_clean[contract_cols]
333 contract_clean = contract_clean.rename(columns={'application_id': 'application_id_res'})
334 contract_clean.to_parquet('store/upload/contractlist_exploded.parquet')
335
336
337 contractpaymentlists = pd.concat(container_payment)
338 contractpaymentlists = contractpaymentlists.sort_values(
339     by=['application_id'],ascending=True
340 )
341 contractpaymentlists = contractpaymentlists.reset_index(drop=True)
342 print('Paymentlist Shape : ',contractpaymentlists.shape)
343 #
344 contractpaymentlists.to_pickle('store/pickle/all_contractpaymentlists_adri_v20250730.pkl'
345 )
346
347 exploded_payment = explode_col(contractpaymentlists,'contractpaymentlists')
348 print('Exploded Contract Shape : ',exploded_payment.shape)
349 payment_clean = exploded_payment.drop_duplicates()
350 payment_clean = payment_clean[payment_cols]
351 payment_clean = payment_clean.rename(columns={'application_id': 'application_id_res'})
352 payment_clean[payment_cols].to_parquet('store/upload/paymentlist_exploded.parquet')
353
354 if __name__ == "__main__":
355     pass
```